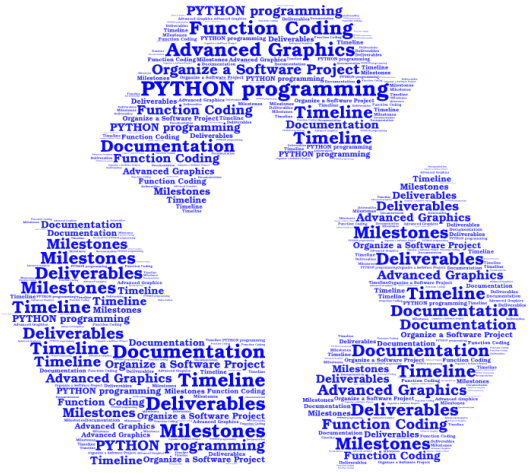
University of Applied Sciences
HOCHSCHULE EMDEN·LEER



Python Programming: Creating Simple Plots

Lecture: Modelling an Simulation

This is a compulsory elective module of

Technology of Circular Economy

(Master of Engineering)

Dr. Uwe Reimer

Senior Scientist

Plotting library

Simple plots are great for generating a quick overview.
There is a special library for Python: Matplotlib.

https://matplotlib.org/stable/

The library needs to be installed. You can do this by using the Windows PowerShell.
Type the following command:

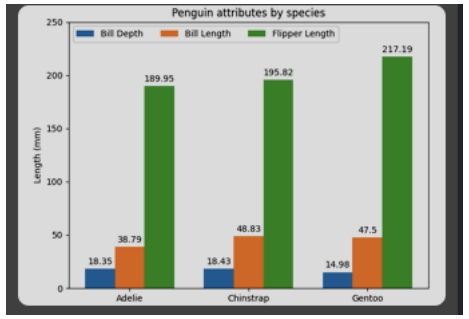
pip install matplotlib

Sometimes, the command above might not work (depending on your computer).
You can try this instead:

python –m pip install matplotlib

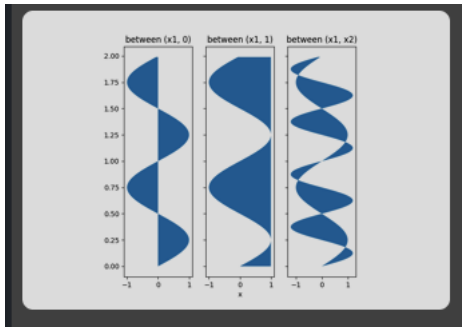
(If this is not working, check *'pip3 install matplotlib*' or '*python3 –m pip install matplotlib*')

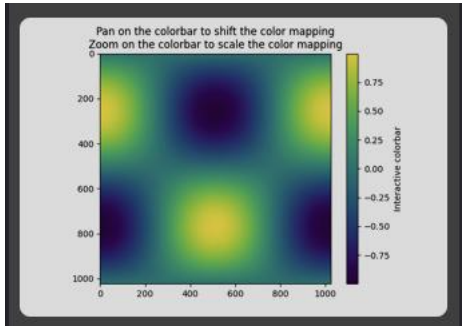
Almost everything is possible ...

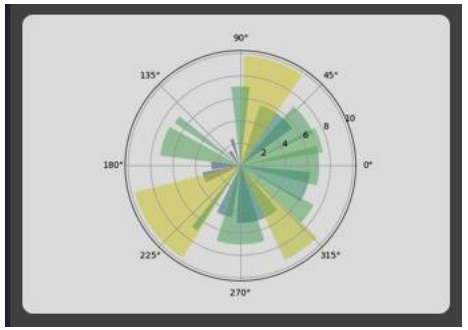


There are many things that can be done. For an overview, check this link:

https://matplotlib.org/stable/gallery/index.html

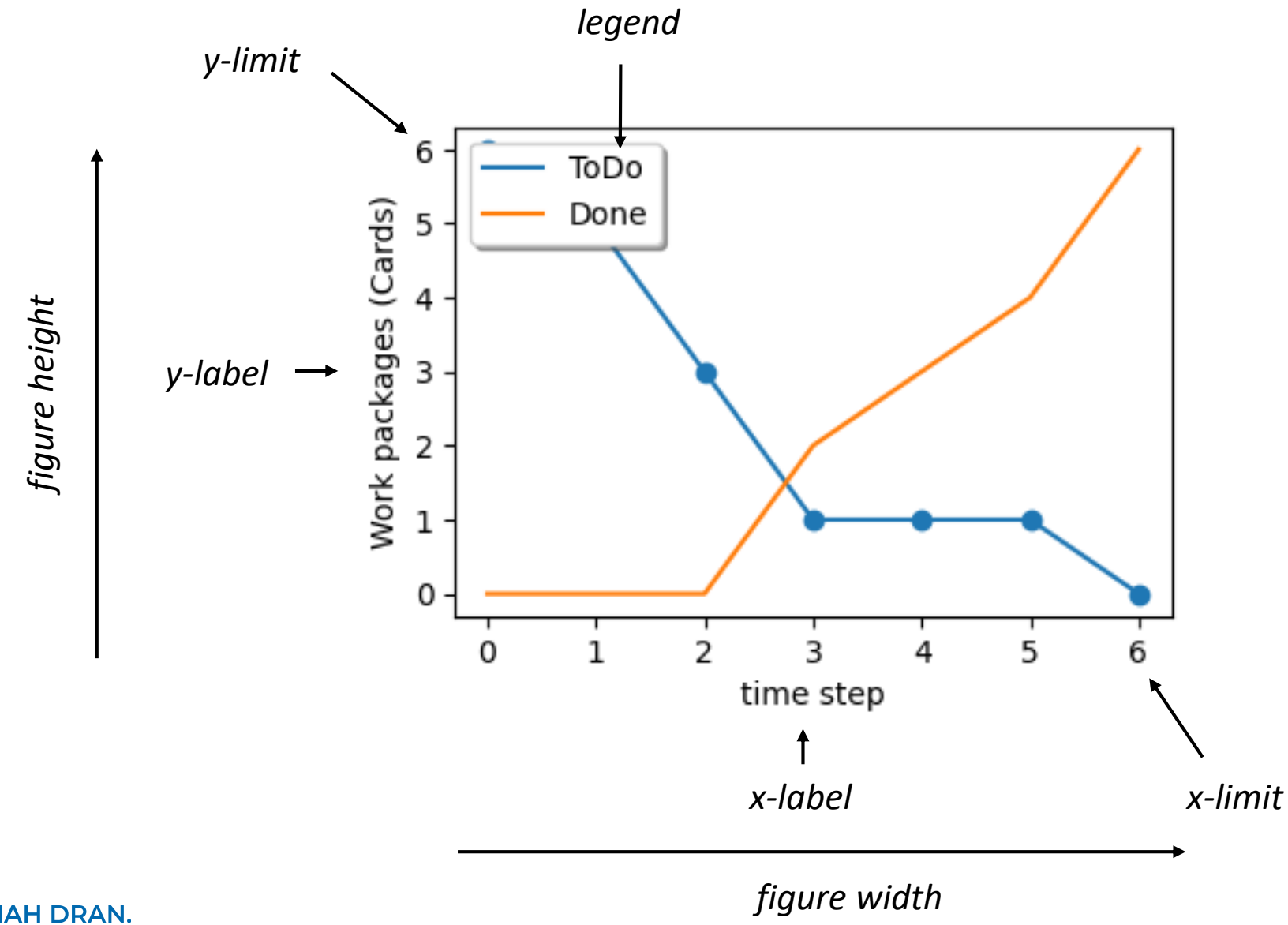






In this lecture, we will cover only 'simple' x-y plots.

Layout of figures



Also important:

- *font sizes*
- *line width*
- *line color (for plots)*

Minimum: plot at level 1

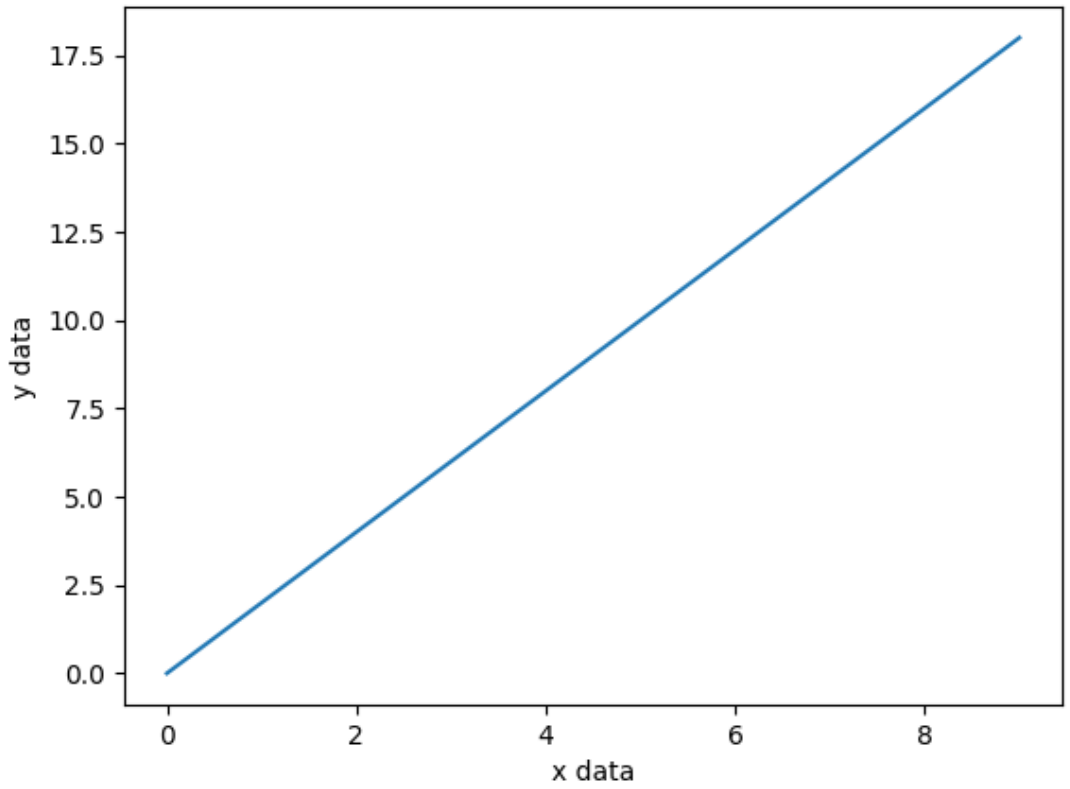
Let us create a minimum plot. The plot will be saved as a *.png file.

What do you need?

1) A set of x-y data
2) Import the Matplotlib library
3) Define plot
4) Name axis
5) Export plot to file

There is a default value for everything that is not specified.

Let us look at our first example –> plot01.py



plot01.py

The library is imported with the name 'plt'. ->

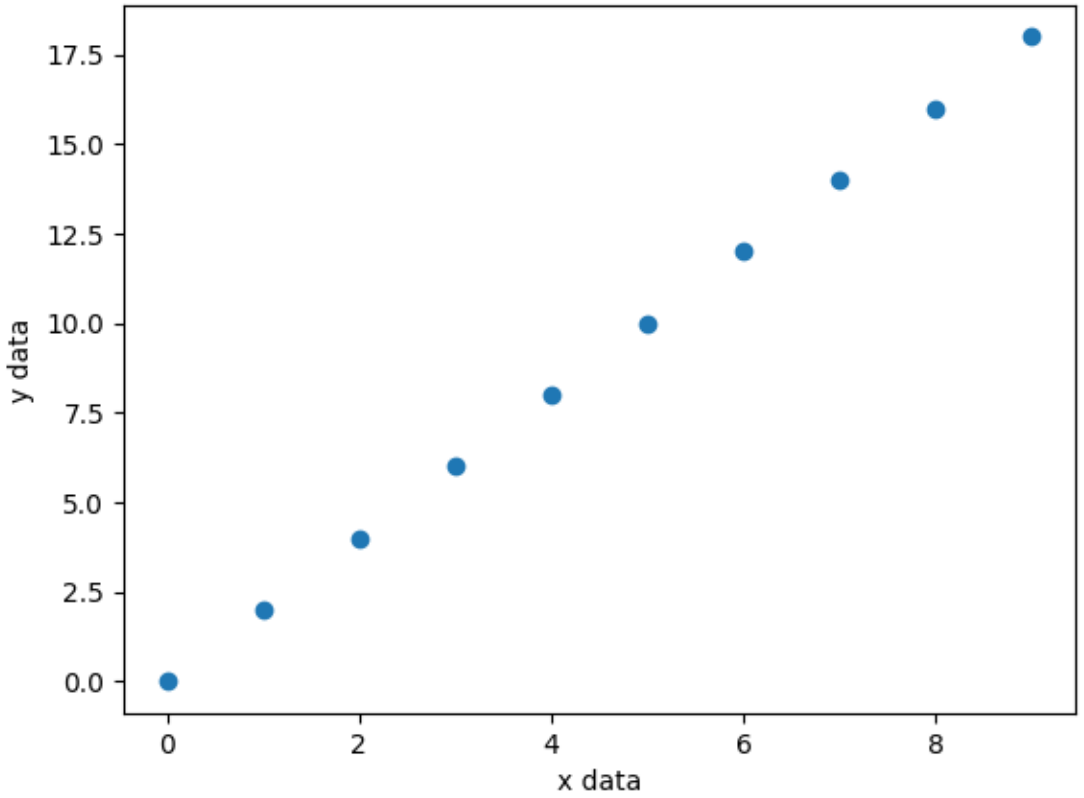
```
 8  import matplotlib.pyplot as plt
 9
10  ### x-y data
11  xdat = range(10)      # a list with integers fro
12  ydat = range(0,20,2)  # a list from zero to 20 w
13
14  ### This section is for plotting
15  pngfile = 'Plot01.png' # file name for output
16
17  plt.plot(xdat, ydat)    # line plot
18
19  plt.xlabel('x data')    # label x-axis
20  plt.ylabel('y data')    # label y-axis
21
22  plt.savefig(pngfile)    # create file for output
```

Everything for the plot starts with 'plt.'

plot02.py

Now, we want to see the data points instead of a straight line.

Therefore, we use a different plot: scatter



plot02.py

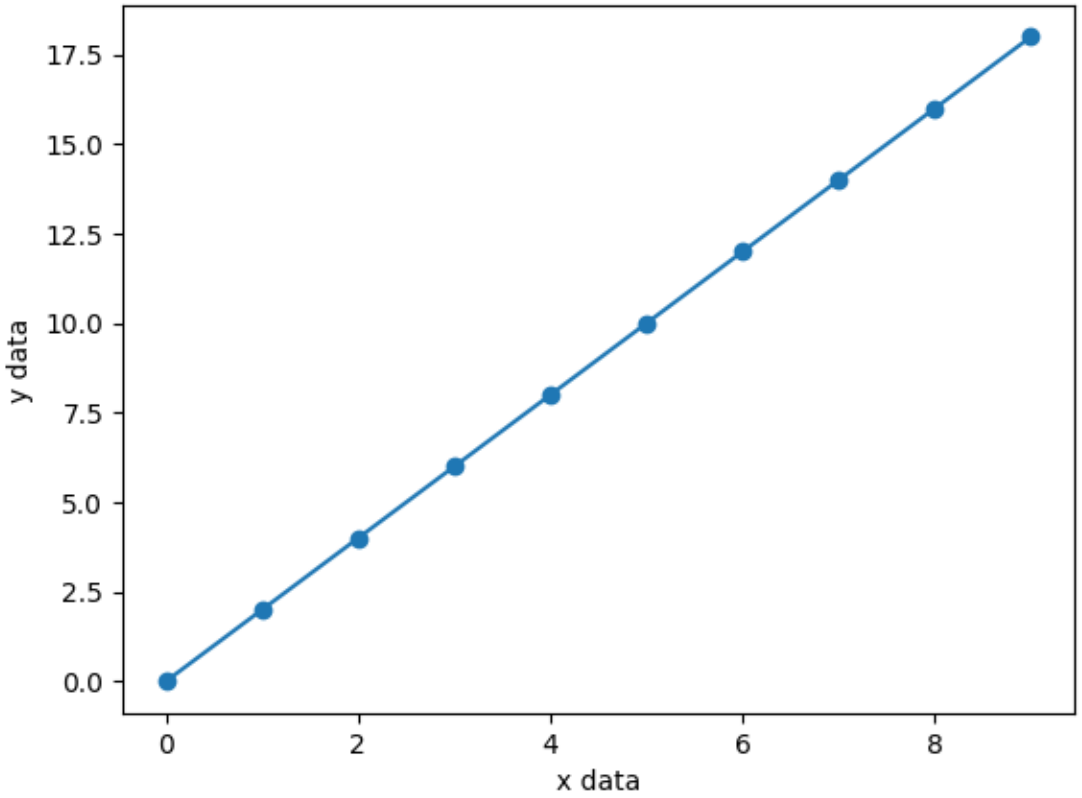
just add this line ->

```
 8  import matplotlib.pyplot as plt
 9
10  ### x-y data
11  xdat = range(10)      # a list with integers from
12  ydat = range(0,20,2)  # a list from zero to 20 wi
13
14  ### This section is for plotting
15  pngfile = 'Plot02.png' # file name for output
16
17  #plt.plot(xdat, ydat)    # line plot
18  plt.scatter(xdat, ydat)    # dots
19
20  plt.xlabel('x data')    # label x-axis
21  plt.ylabel('y data')    # label y-axis
22
23  plt.savefig(pngfile)    # create file for output
```

plot03.py

You can combine several plots.
Here, we use a line plot and a scatter plot to show the same data set.

Python recognizes that it is the same data set -> the same color is chosen.
If you combine different data sets, different colors will be chosen.



plot03.py

line plot and -> 17
scatter plot -> 18
in the same figure

```
 8  import matplotlib.pyplot as plt
 9
10  ### x-y data
11  xdat = range(10)       # a list with integers fro
12  ydat = range(0,20,2) # a list from zero to 20 w
13
14  ### This section is for plotting
15  pngfile = 'Plot03.png' # file name for output
16
17  plt.plot(xdat, ydat)    # line plot
18  plt.scatter(xdat, ydat) # dots
19
20  plt.xlabel('x data')    # label x-axis
21  plt.ylabel('y data')    # label y-axis
22
23  plt.savefig(pngfile)    # create file for output
```

Add some control - plot at level 2

Minimum plot – level 1

- A set of x-y data
- Import the Matplotlib library
- Define plot
- Name axis
- Export plot to file

Now, we add

- Size of figure
- Legend – for using several data sets
- Placement of legend
- Scaling the axis

-> see 'Plot04.py'

You can download this from the Moodle course.

As example, we will use the data from the Kanban metrics from a previous lecture.

Plot04.py

set export file name

set width and height

note the label

create legend

optimize placement

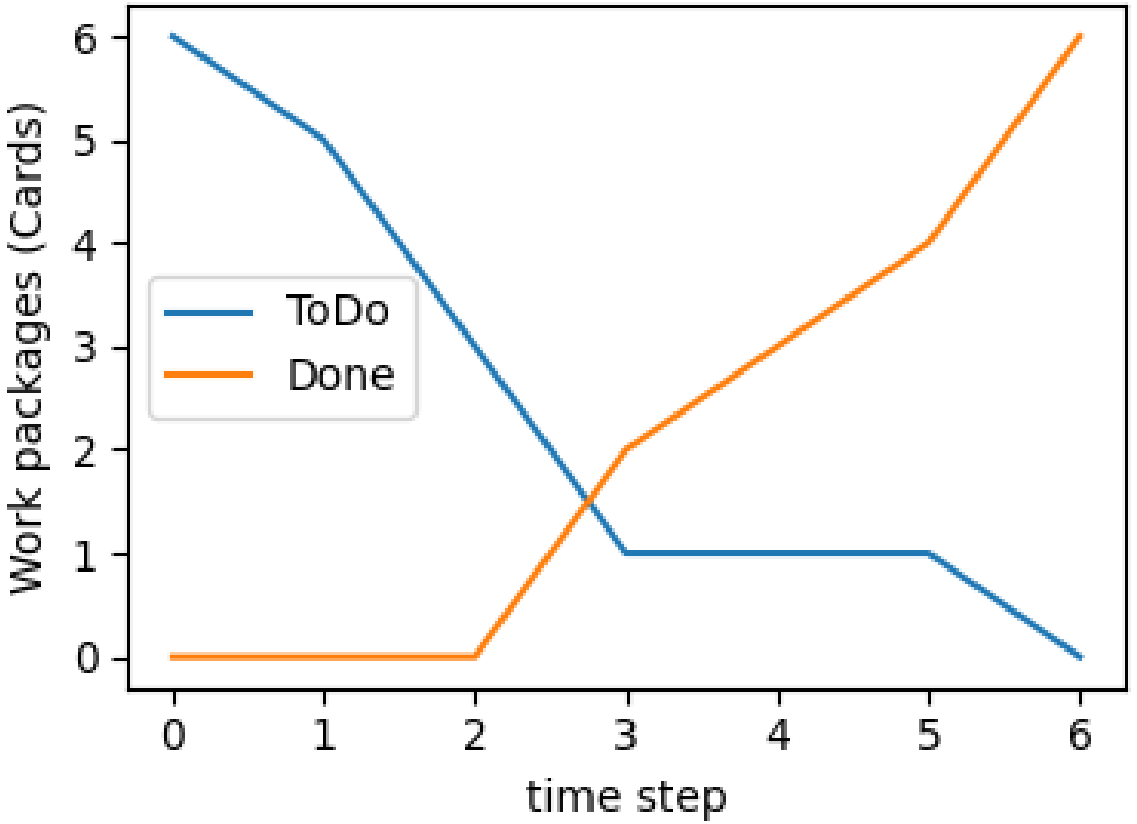
```
18  pngfile = 'Plot04.png'          # file name for
19
20  fwidth = 10                     # figure width
21  fwidth = fwidth / 2.54            # conversion
22  fheight = fwidth * 3/4          # using a ratio
23  plt.subplots( figsize=( fwidth,fheight ) )
24
25  plt.plot(xdat, y1dat, label='ToDo') # line
26  plt.plot(xdat, y2dat, label='Done') # line
27
28  plt.xlabel('time step')                 # label
29  plt.ylabel('Work packages (Cards)') # label
30
31  plt.legend()                    # legend - need
32  plt.tight_layout()              # optimize plac
33  plt.savefig(pngfile)            # create file f
```

Plot04.py

Now, let's add the data points as scatter plot. We will change the style of the points.

Remember, how to add the scatter plot:

```
plt.scatter(xdat, y1dat)
plt.scatter(xdat, y2dat)
```

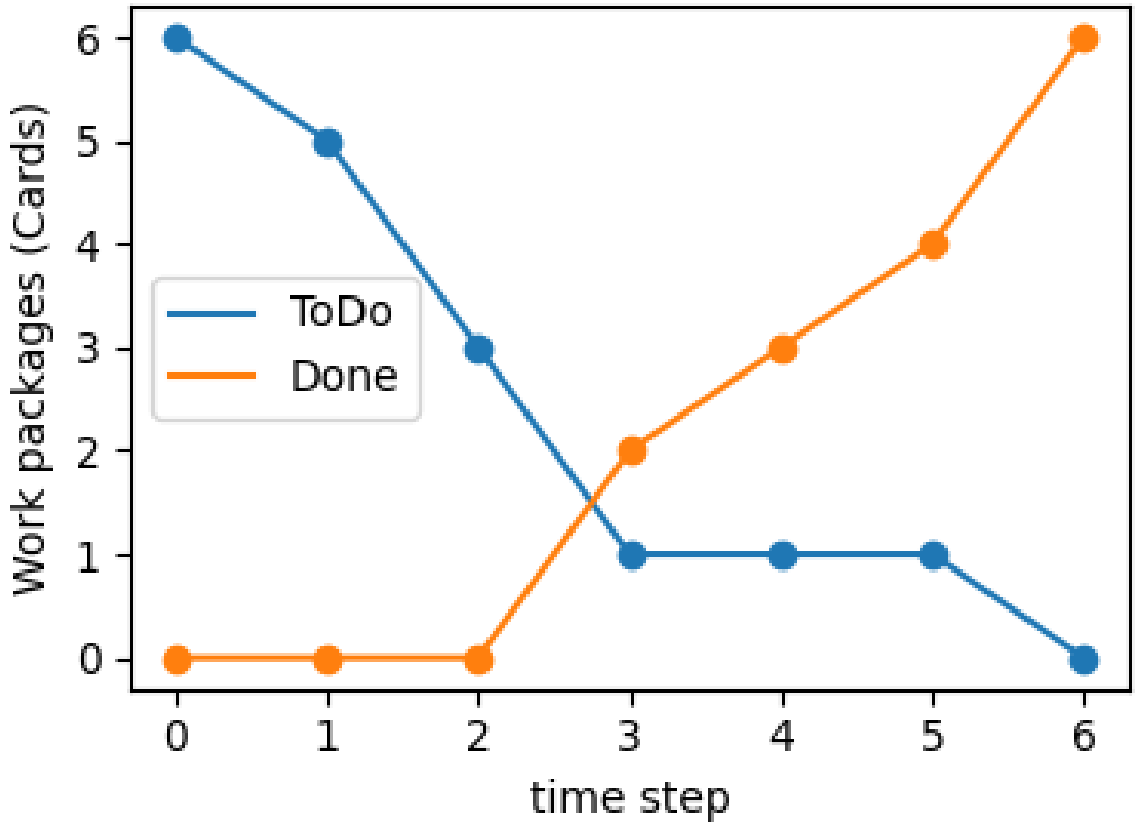


Plot05.py

You can change the style of the data points.

```
plt.scatter(xdat, y1dat, marker='D')
plt.scatter(xdat, y2dat, marker='s')
```

●	.	◆	D
■	s	▼	v
★	*	▲	^
✕	x	◀	<
✖	X	▶	>

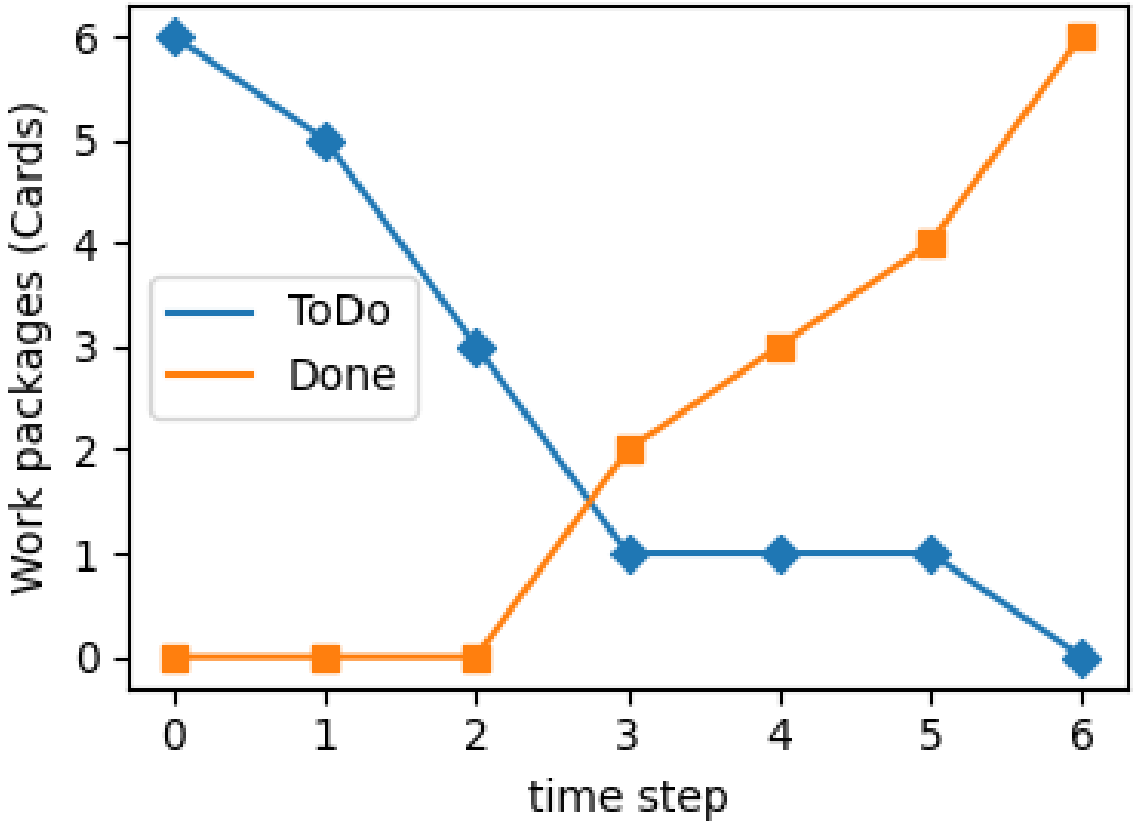


Plot06.py

Now, let's change the scale of the x- and y-axis. Both should be from -0.5 to 10.

Add the following code after the plot and before the 'plt.savefig'.

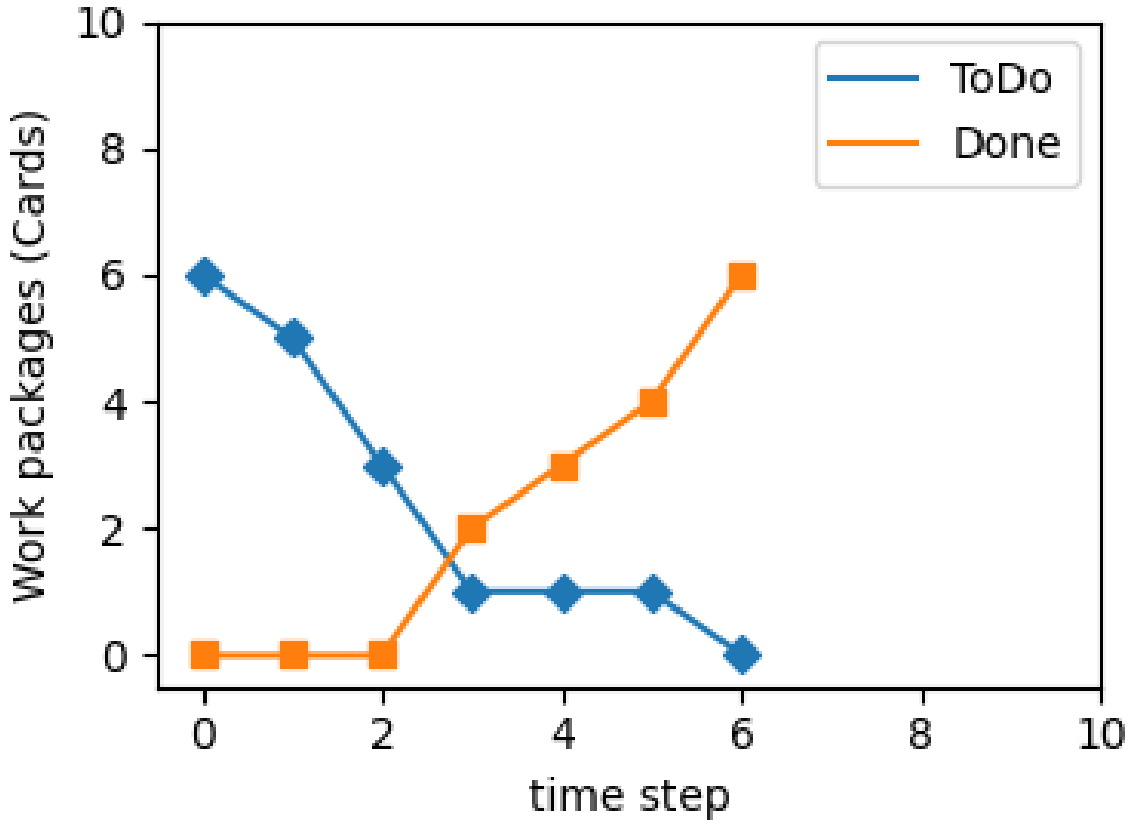
```
plt.xlim(-0.5,10)
plt.ylim(-0.5,10)
```



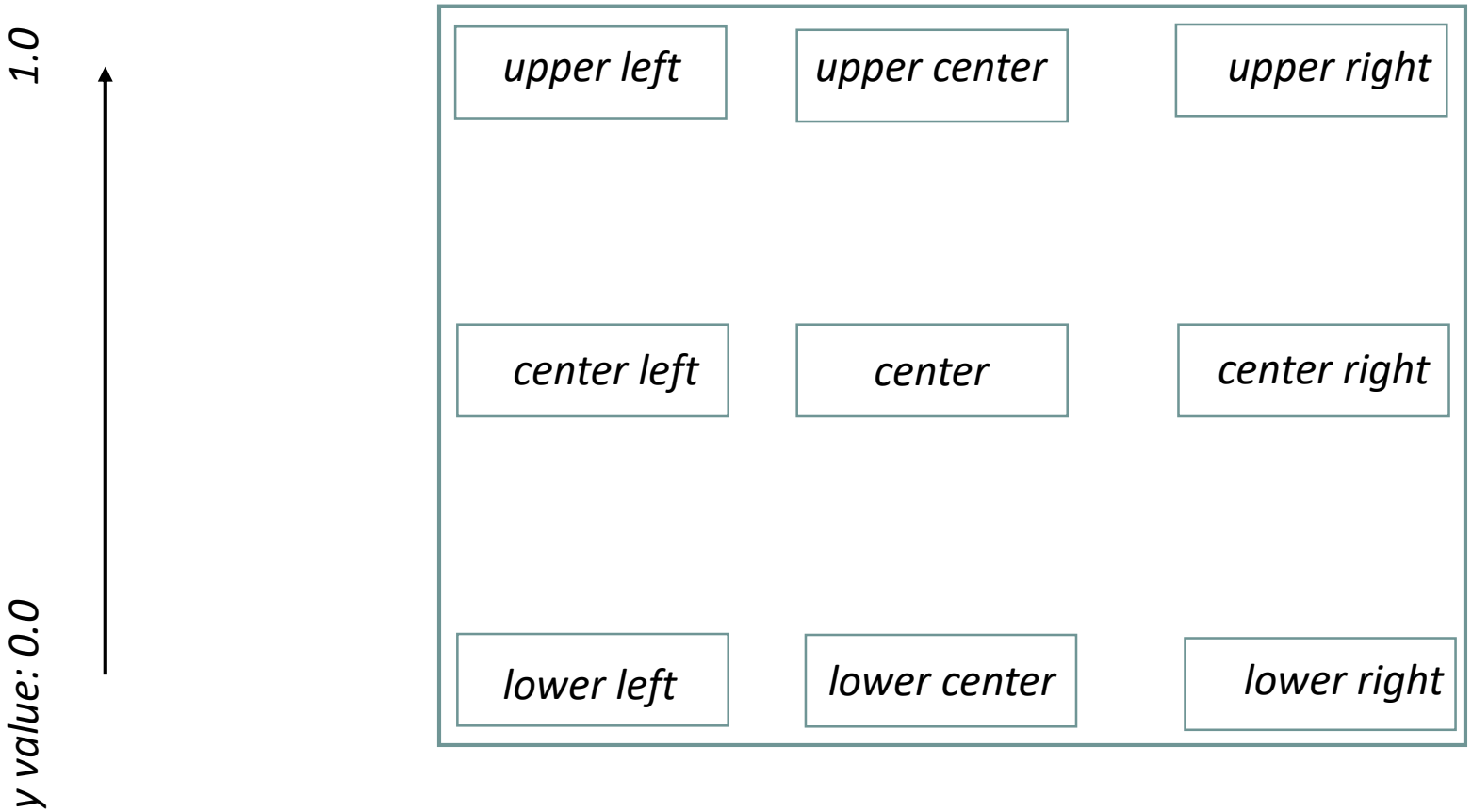
Plot07.py

Note, how the placement of the legend changes automatically.

If you want to have the legend at a certain position, you need to define it.



Placement of legend

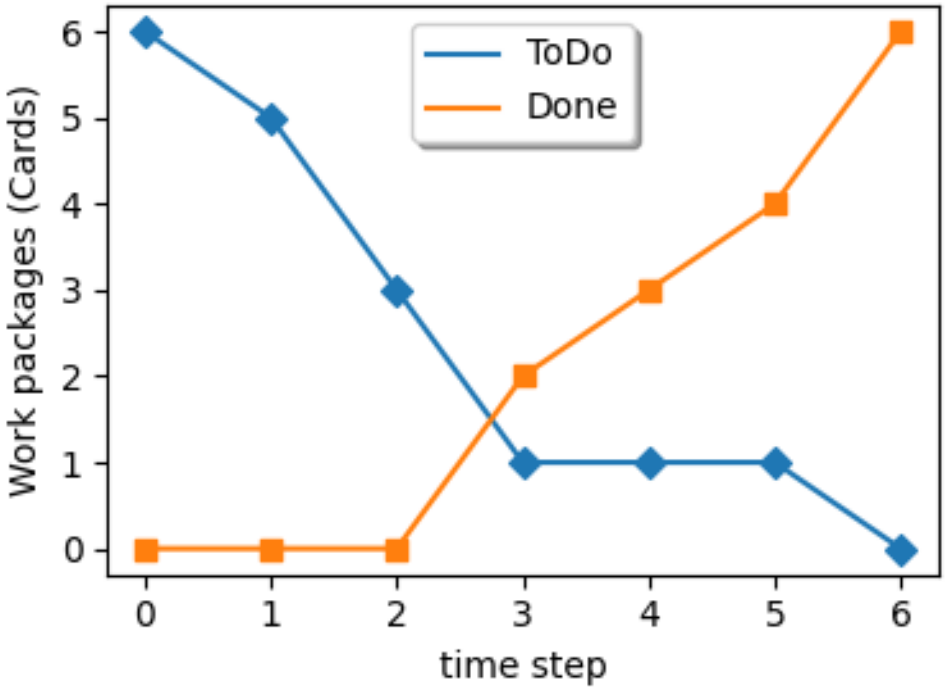


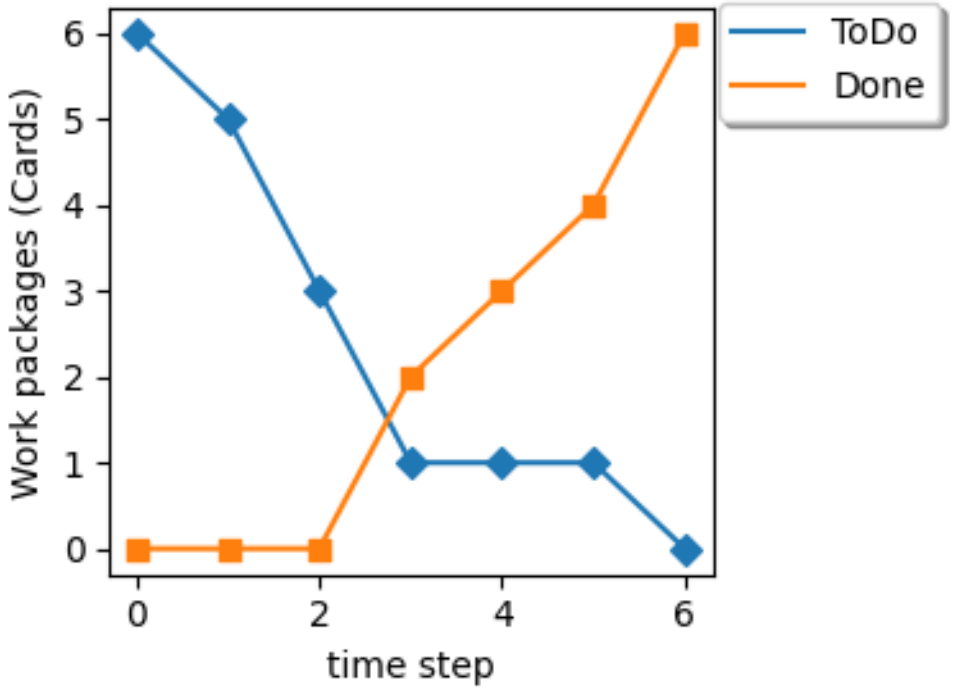
legend at (1.01, 0.7)

Also possible:

Free placement with (x,y) position.
Figure sides range from 0.0 to 1.0

Plot08.py





Left – an example for placement in the center

plt.legend(loc='upper center', shadow=True)

Right – an example for placement outside

plt.legend(loc=(1.01, 0.8), shadow=True)

Add more control - plot at level 3

Minimum plot – level 1

- A set of x-y data
- Import the Matplotlib library
- Define plot
- Name axis
- Export plot to file

Examples: plot01 .. plot03.py

Plot level 2

- Size of figure
- Legend – for using several data sets
- Placement of legend
- Scaling the axis

Examples: plot04 .. plot08.py

Plot level 3

- Add a second y-axis
- Annotations
- Custom font sizes
- Custom colors

Examples: next slides

Adding a second y-axis

First, we create two sets of x-y data.

quadratic function:
$y1 = x^2$

linear function:
y2 = x

```
 8  import matplotlib.pyplot as plt
 9
10  ### generate curves
11  xdat = range(10)
12  y1dat = []
13  y2dat = []
14
15  for x in xdat:
16      a = x * x          # Returns square of x
17      y1dat.append(a)
18      a = x
19      y2dat.append(a)
20
```

Adding a second y-axis / plot09.py

import matplotlib.pyplot as plt

Before, we used the following code to initialize the figure.

plt.subplots(figsize=(fwidth,fheight))

Here, everything specific for the plot is referenced by 'plt.'

Now, we use a different call:

fig, ax1 = plt.subplots(figsize=(fwidth,fheight))

Here, we get more control.
All things for the plot are referenced by 'fig.'
and all things specific for axis is referenced by 'ax1.'

Adding a second y-axis / plot09.py

different call of function (we get a separate handler for axis)

```
27  fig, ax1 =plt.subplots( figsize=( fwidth,fheight ) )
28
29  ax2 = plt.twinx(ax1)
30
31  ax1.set_xlabel('x-value')
32  ax1.set_ylabel('y = x$^2$')
33  ax2.set_ylabel('y = x')
34
35  ax1.tick_params(axis='both')
36
37  ax1.plot(xdat, y1dat, label='left side')
38  ax2.plot(xdat, y2dat, label='right side')
```

← define second axis, sharing the same x

← activate ticks on both sides

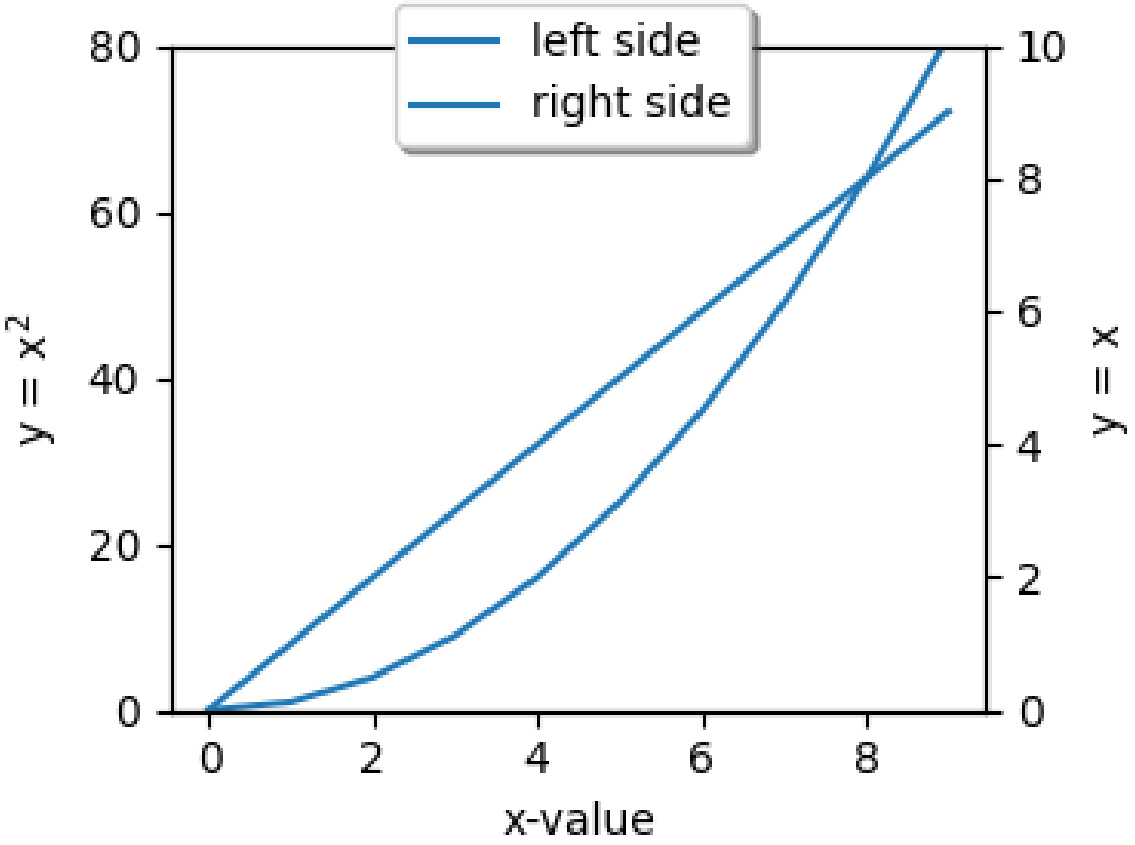
plot is specific to an axis

Plot09.py

Now, we have the following problem that both plots are in the same color.
Why? Because both axis are treated separately. For each axis this is the first plot, hence it gets the same default color.

We need to change the color of the plot.

ax1.plot(xdat, y1dat, label='left side', color='red')
ax2.plot(xdat, y2dat, label='right side', color='blue')

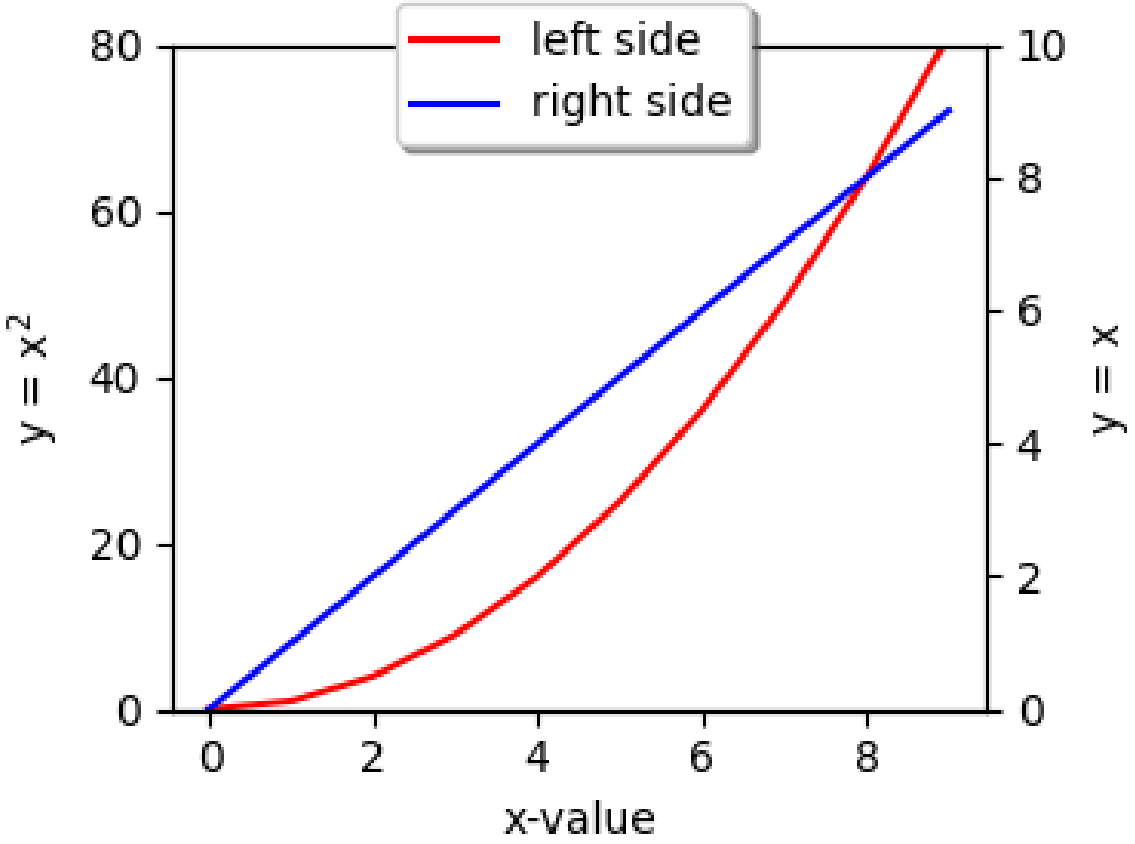


Plot10.py

This is better. But how do you know which plot belongs to which axis? The legend is not very helpful. Let us add some arrows.

ax1.arrow(3.8, 15.0, -1.0, 0.0, head_width=1.3, head_length=0.2)
ax1.arrow(8.3, 66.0, 1.0, 0.0, head_width=1.3, head_length=0.2)

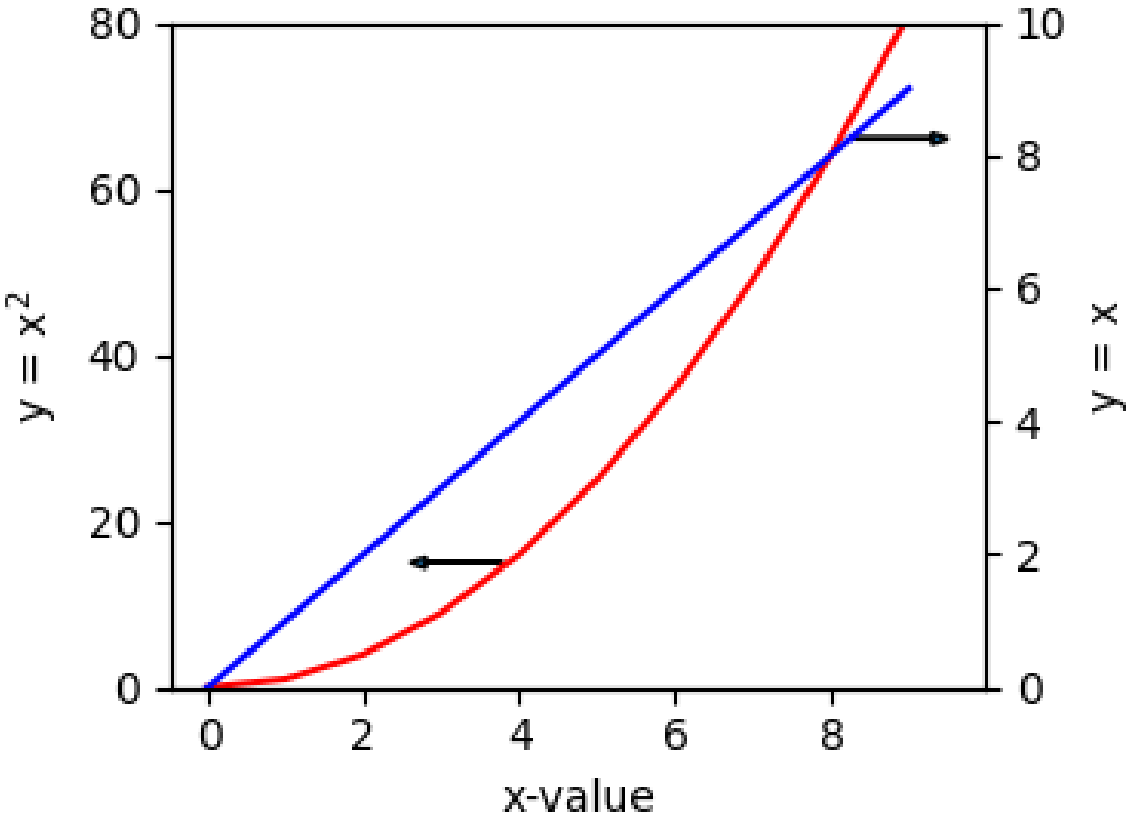
- specification is for a certain axis, here 'ax1'
- position is given in axis coordinates x, y, dx, dy
- head width and length of arrow are given in points



Plot11.py

This is much better.

Figure size
Imagine, you need a figure with a higher resolution (for poster or a scientific paper).
This is easy, just increase the size of the figure to 30 x 22 cm and you have a nice, big picture.



Plot11.py

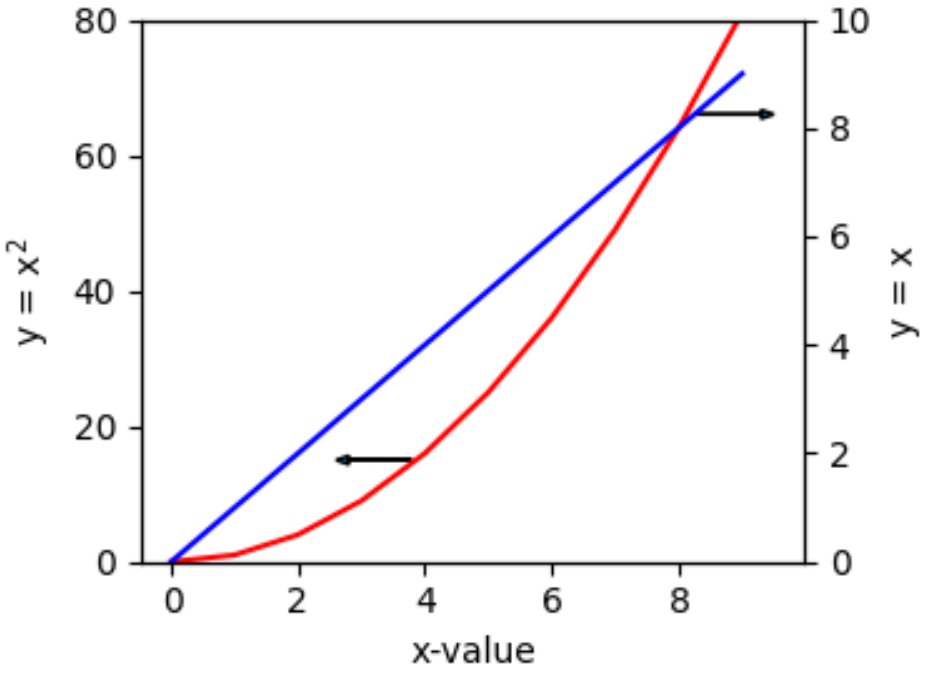


Figure size is 10 x 7 cm
(scaled to height of 10 cm)

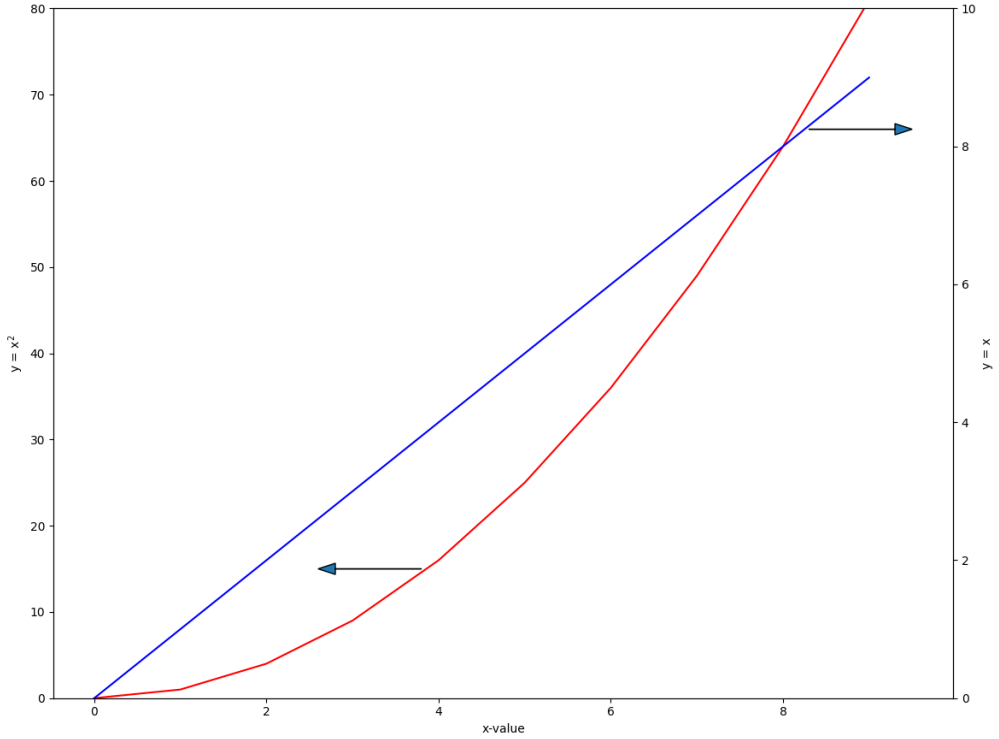
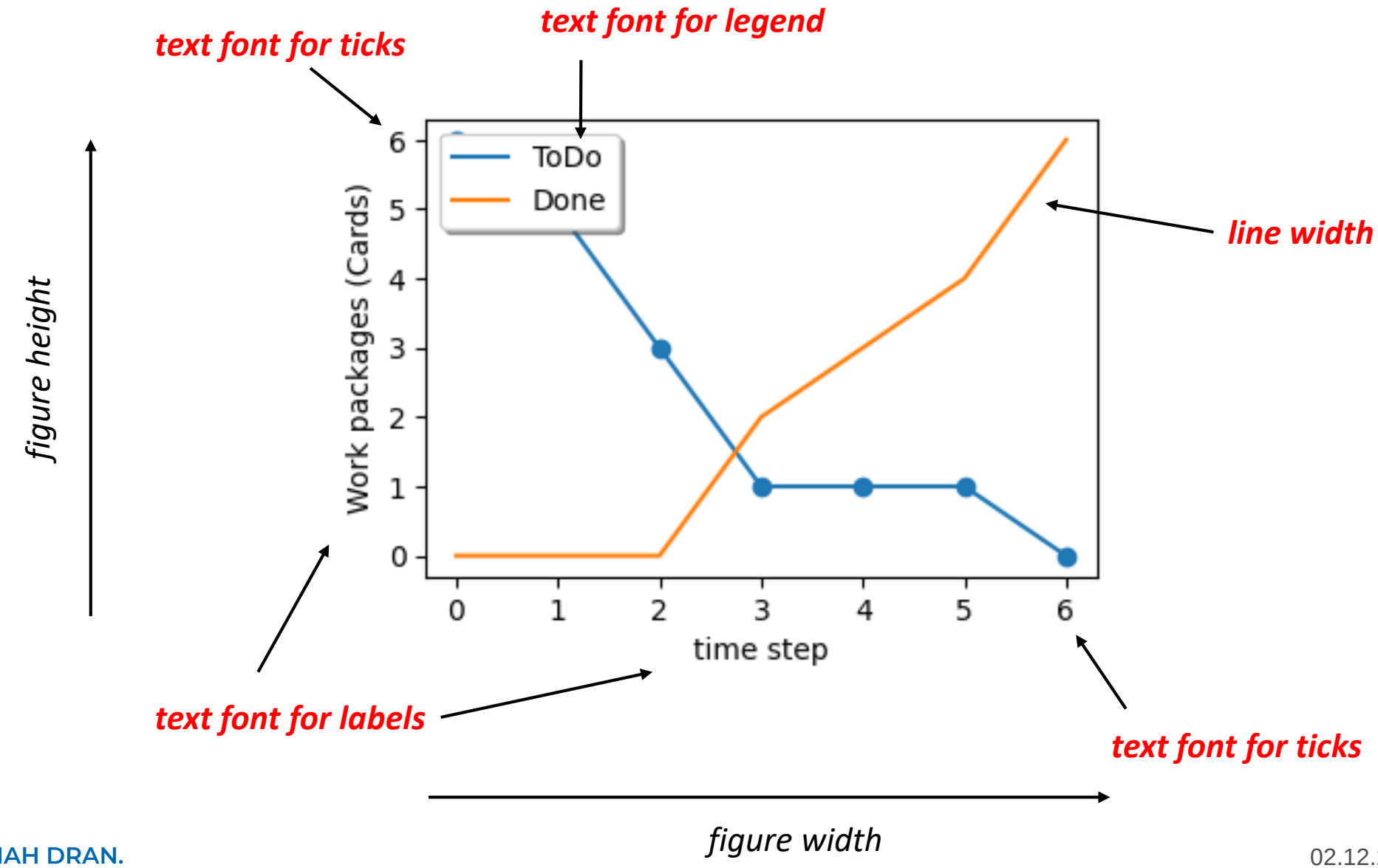


Figure size is 30 x 22 cm
(scaled to height of 10 cm)

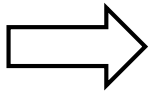
The right picture has much higher resolution, but text fonts are to small and lines are very thin.
These things need to be scaled also (instead of using the default values).

Layout of figures



Plot12.py

We define two different font sizes:
one for tick label and
one for axis label.

Figure size is 10 x 7 cm  Figure size is 30 x 22 cm

```
30  # here is the definition of font sizes # ------------
31  myfontsize = 32      # axis title
32  myfontsize2 = 28     # everything else
33  # apply sizes to graph
34  plt.rcParams['font.size'] = myfontsize2 # for legend
35  plt.xticks(fontsize= myfontsize2)
36  # end of the definition of font sizes # ------------
37
38  ax1.set_xlabel('x-value', fontsize= myfontsize)
39  ax1.set_ylabel('y = x$^2$', fontsize= myfontsize)
40  ax2.set_ylabel('y = x', fontsize= myfontsize)
41
42  ax1.tick_params(axis='both', labelsize= myfontsize2)
43  ax2.tick_params(axis='both', labelsize= myfontsize2)
```

Plot12.py

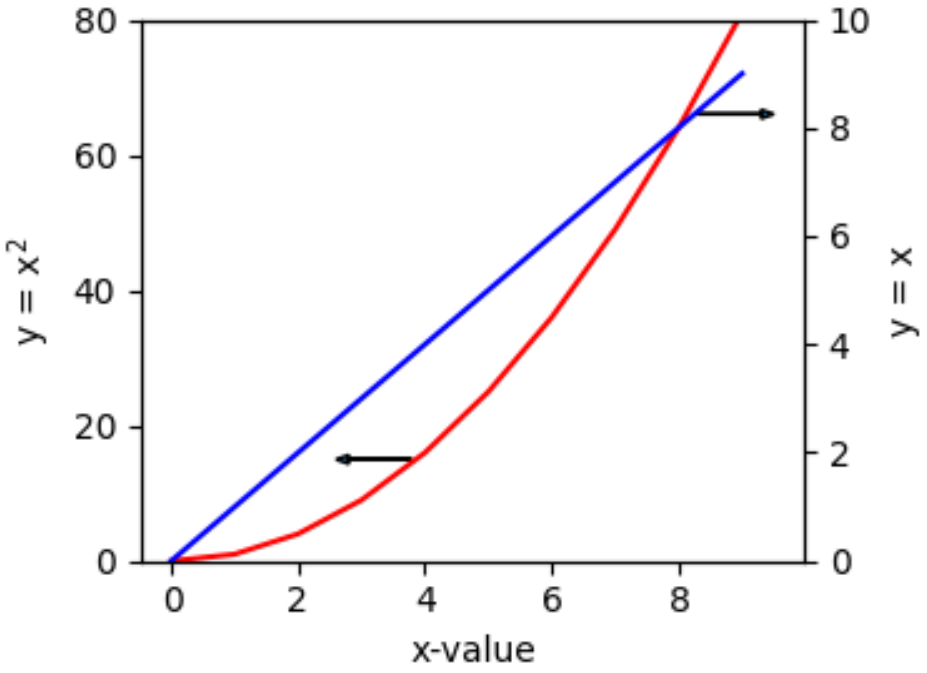


Figure size is 10 x 7 cm
(scaled to height of 10 cm)
(plot11.py)

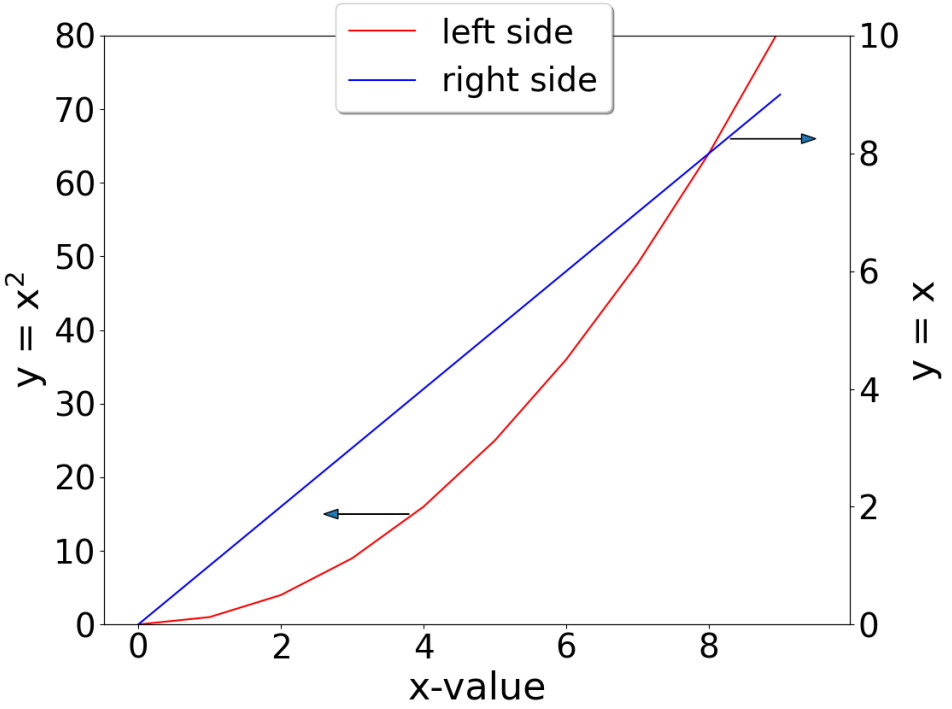


Figure size is 30 x 22 cm
(scaled to height of 10 cm)

Now, font sizes are OK, but lines are too thin.

Plot13.py

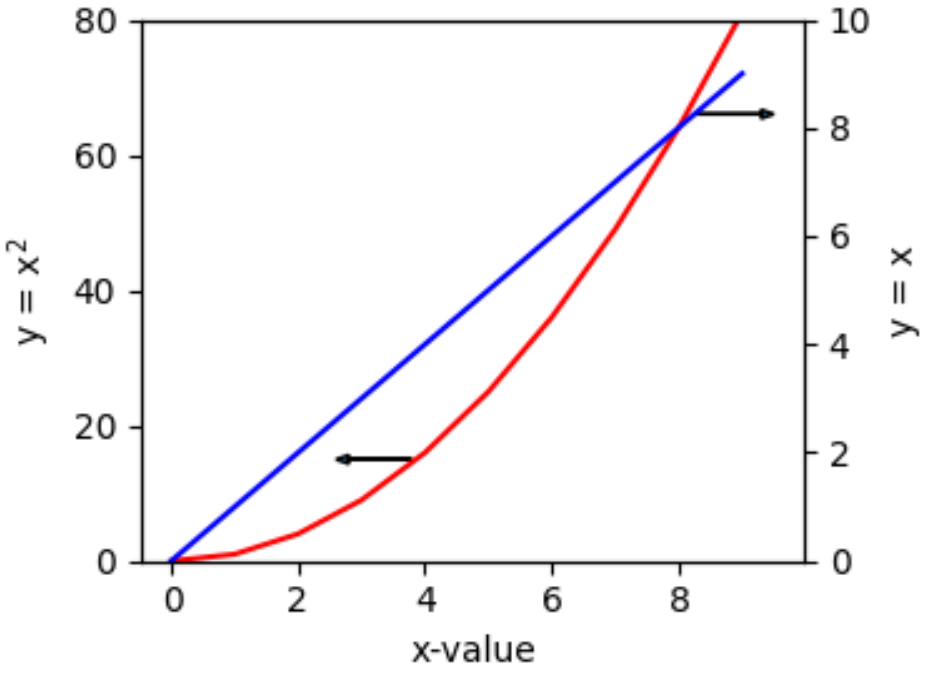


Figure size is 10 x 7 cm
(scaled to height of 10 cm)

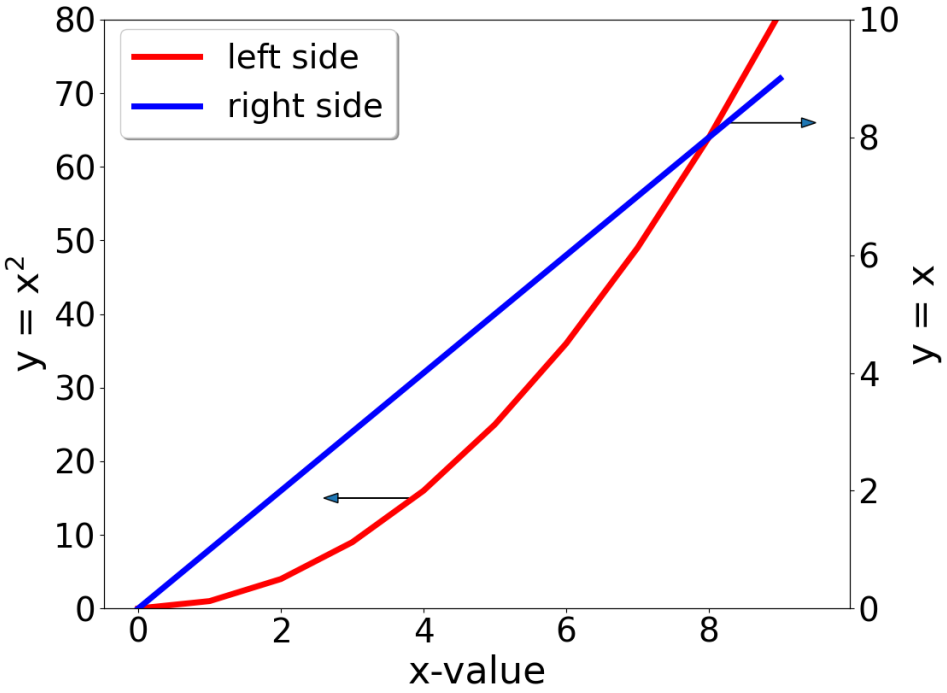


Figure size is 30 x 22 cm
(scaled to height of 10 cm)

Specification for lines:

ax1.plot(xdat, y1dat, label='left side', color='red', linewidth=5)
ax2.plot(xdat, y2dat, label='right side', color='blue', linewidth=5)

Short summary – adjusting plot size

As you add more features, like

- Change size of figure
- Customize font sizes
- Add a second y-axis

... the programming becomes more demanding.

➢ Therefore, you should keep the plot as simple as possible – this makes it easier for others to understand your data.

➢ The text in the plot should be the same as the smallest text in your report (on that page).

➢ Create an example plot for yourself that you find useful. Use this reference to create similar plots throughout your work. It makes your work more consistent.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Custom colors

For a quick overview it is sufficient to use the default colors.
If you want to have a nice graph, you may want to choose your own colors.

In Matplotlib there are several ways to define colors.
We will use two notations here:

Named colors according CSS definition
https://matplotlib.org/stable/gallery/color/named_colors.html
Example: 'red' for red

Hex RGB values
You can find many color tables with hex RGB values in the internet.
(See picture to the right.)
Example: *'#cc0000' for red*

Multiple plots – plot14.py

Colors are needed, if you have several line plots to distinguish.
Let's create a set of three x-y data sets.

For x-values, we will use the **range()** function. It creates a list of integers.
Here from 0 .. 9.

For y-values we will use the **random()** function.
The random() function is loaded from the module 'random'.
By default, random() returns a floating point value between 0 and 1.

You can download this example from the Moodle course.

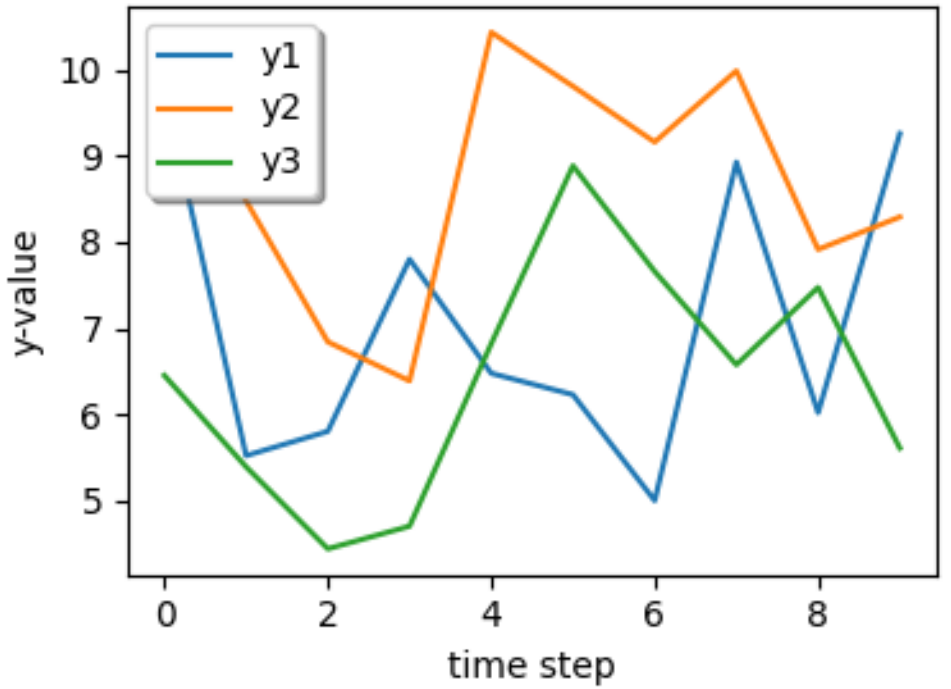
```
 8  import matplotlib.pyplot as plt
 9  import random
10
11
12  xdat = range(10)
13  y1dat = []
14  y2dat = []
15  y3dat = []
16
17  for x in xdat:
18      a = 5.0 + 5.0 * random.random()
19      y1dat.append(a)
20      a = 6.0 + 5.0 * random.random()
21      y2dat.append(a)
22      a = 4.0 + 5.0 * random.random()
23      y3dat.append(a)
```

Plot14.py

This plot uses default colors.
Now, we add our own colors according to the CSS names.

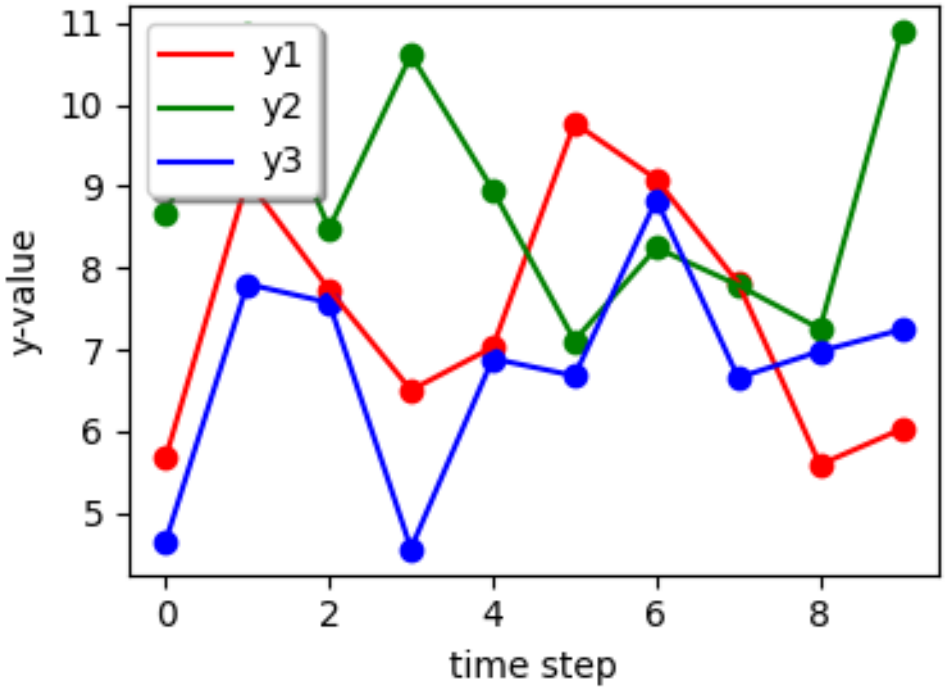
y1 = red
y2 = green
y3 = blue

Also, we add the data point as scatter plots.
Note, that you need to specify colors also in the scatter plots.



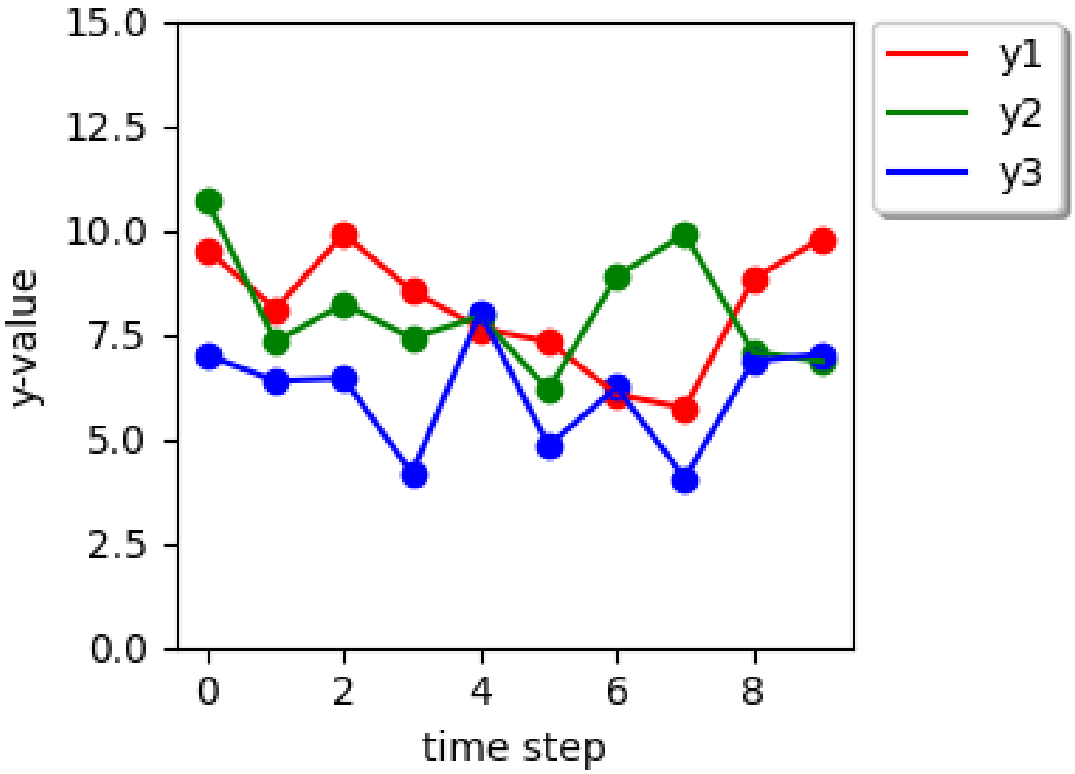
Plot15.py

Now, let's scale the y-axis to start from zero to 15
(to see, if the differences are really that big)
and place the legend outside.



Plot16.py

Now, let's scale the y-axis to start from zero to 15
(to see, if the differences are really that big)
and place the legend outside.



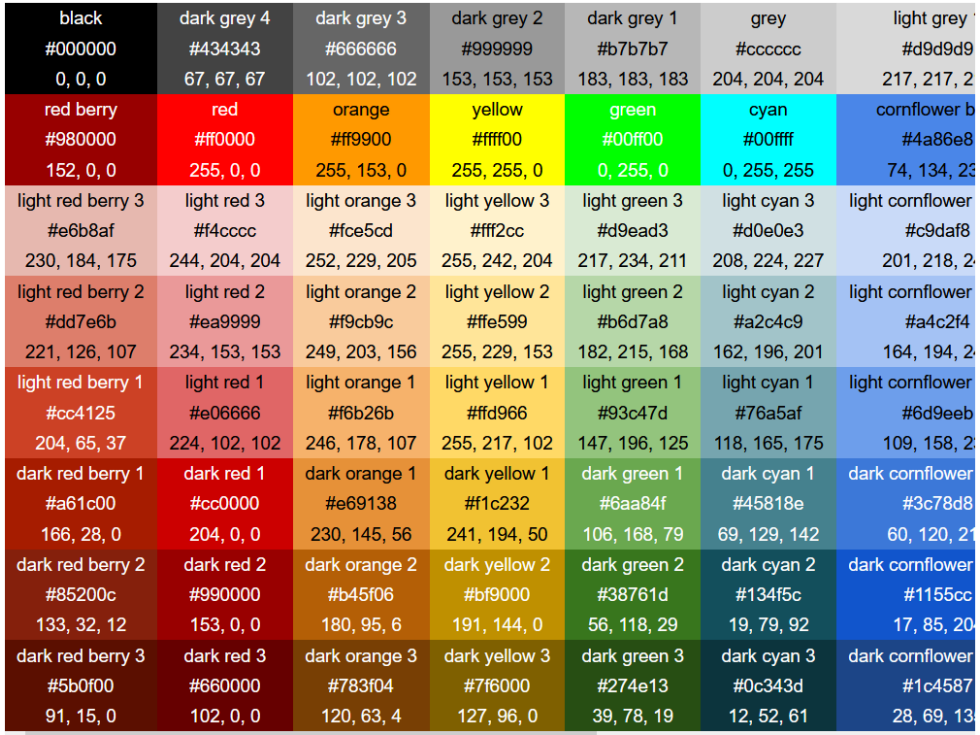
University of Applied Sciences
HOCHSCHULE EMDEN·LEER

A list of custom colors

Here is an example color definition. For the lecture we create a list where you can choose from.
(You can download 'plot17.py' from the Moodle course.)

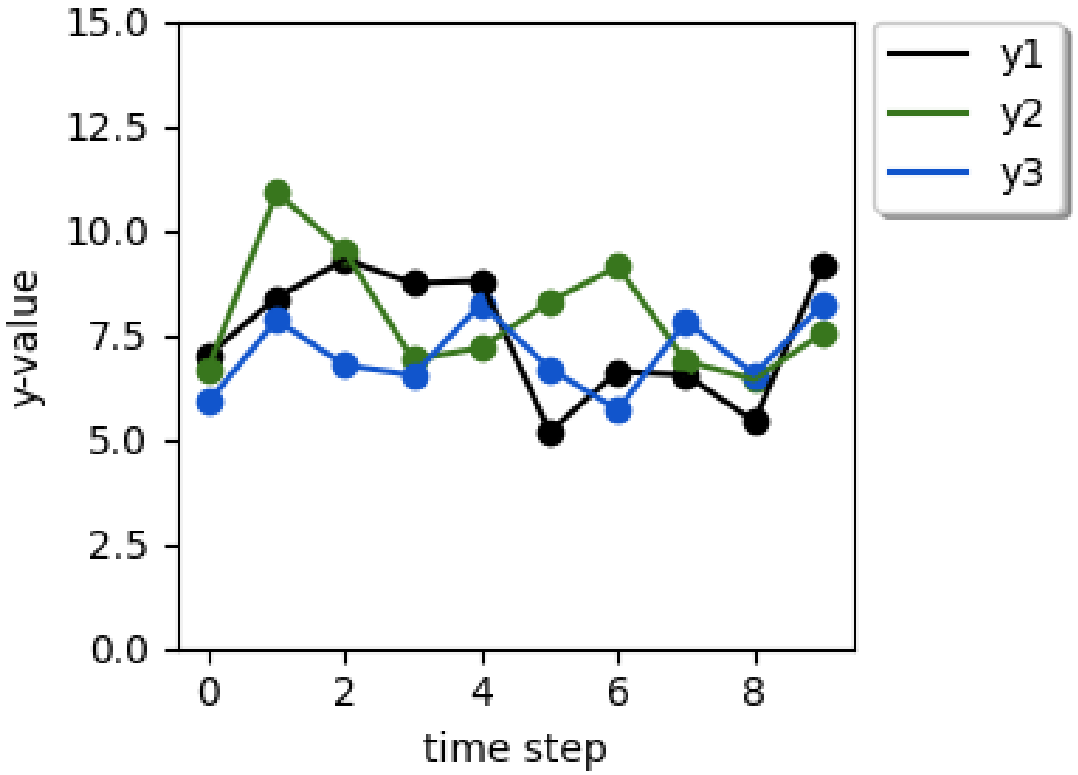
In your program, you can just choose the position in a list instead of typing hexadecimal numbers.

```
#000000         # 0 black
#38761d         # 1 green
#1155cc         # 2 blue
#cc0000         # 3 red
#ff9900         # 4 orange
#ffff00         # 5 yellow
#00ff00         # 6 neon green
#00ffff         # 7 cyan
#ff00ff         # 8 magenta
00900ff         # 9 purple
#0000ff         # 10 blue 2
#cccccc         # 11 grey light
#666666         # 12 grey dark
```



Plot17.py

Colors from the list.



Add more control - plot at level 3

Minimum plot – level 1

- A set of x-y data
- Import the Matplotlib library
- Define plot
- Name axis
- Export plot to file

Examples: plot01 .. plot03.py

Plot level 2

- Size of figure
- Legend – for using several data sets
- Placement of legend
- Scaling the axis

Examples: plot04 .. plot08.py

Plot level 3

- Add a second y-axis
- Annotations
- Custom font sizes
- Custom colors

Examples: plot09 .. plot17.py

Example from the lecture

Let's make a working example, how to plot data which you have from a file.
In a previous lecture we calculated the monthly solar power generation for the cities of Emden and Kochi.

You can download this example from the Moodle course 'Example-solar.zip'.

If you want to import text files, they must be in 'UTF-8' format. This is the text standard format, if the file is generated by Python (open file in write mode) or if you export data from EXCEL.
If you print data to the screen in Python and redirect the output to a text file, then WINDOWS creates another format 'UTF-16 LE-BOM'. You can check this with the NotePad++ program under 'Coding' or 'Codierung' and change it to 'UTF-8'.

Let's look at the program 'plot-solar1.py'. Commonly, we have to follow these steps:

- import data from file
- create plot

Plot-solar1.py

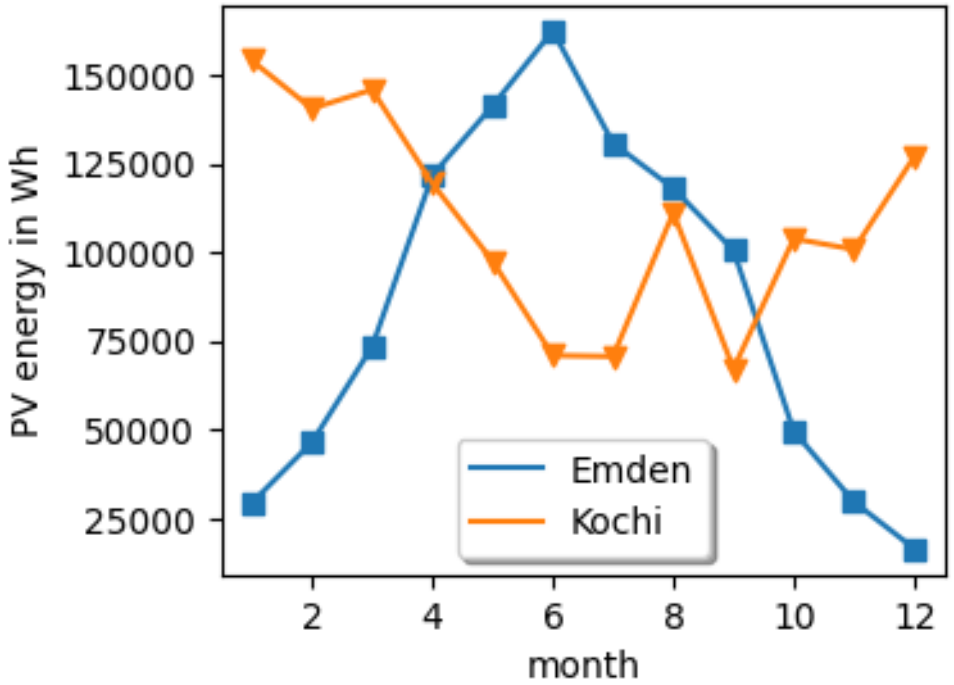
The monthly solar power generation for the cities of Emden and Kochi for the year 2023.

Now, let's do something fancy.
We use only the data for Emden und we will create a plot, which fills the area between the line and the x-axis.

The command for such a plot is:

plt.fill_between(emdenx, emdeny, 0, color='yellow')

x-values fill between

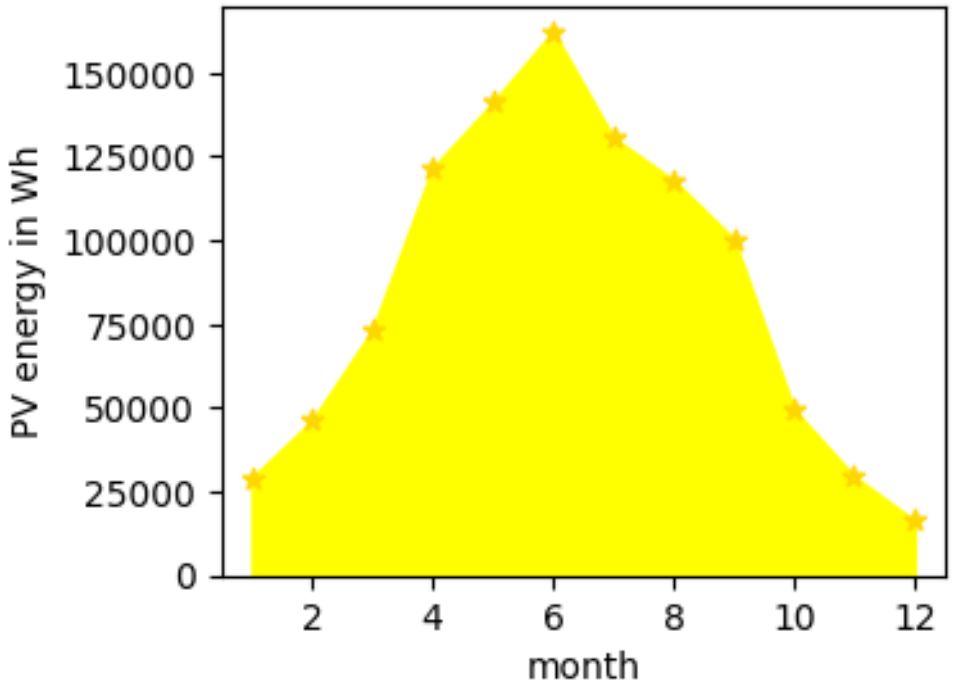


Plot-solar2.py

The monthly solar power generation for the city of Emden the year 2023.

plt.fill_between(emdenx, emdeny, 0, color='yellow')

x-values fill between



Summary

Matplotlib is a Python module for creating graphs. It needs to be installed, it is not part of the standard distribution.

In this lecture, you have seen how to:

- create simple x-y plots
- customize colors
- adjust limits for axis

Advanced topics have been:

- resizing the figure: adjusting text font and linewidth
- adding a second y-axis
- adding arrows for annotation

Hint for beginners:
Try to keep it simple. You can spend easily several hours on graphics.
You can use the examples from this lecture to create your own plots.
For more information, check the examples under
https://matplotlib.org/stable/gallery/index.html